

CONFIDENTIAL — TRAINING ASSESSMENT

WEB APPLICATION SECURITY ASSESSMENT REPORT

Target Application:

OWASP Juice Shop — User Management System

Prepared by

Bushra

Cybersecurity Intern

Week 1 — Security Assessment Task
20 June 2026

Document Information

Application Tested	OWASP Juice Shop (v20.0.0)
Testing Type	Manual + Automated (Black-box)
Test Environment	Local instance — http://localhost:3000
Tools Used	OWASP ZAP 2.17.0, Browser Developer Tools (Chrome), Manual payload injection
Tester	Bushra — Cybersecurity Intern
Assessment Period	Week 1 — Security Assessment
Report Date	20 June 2026

Executive Summary

This report documents the findings of a Week 1 security assessment conducted against the OWASP Juice Shop application, an intentionally vulnerable web application used for security training purposes. The assessment combined manual testing techniques with an automated vulnerability scan using OWASP ZAP.

A total of six findings were identified, ranging from Critical to Low severity. The most significant issue is a SQL Injection vulnerability in the login form that allows complete authentication bypass, granting administrative access without valid credentials. A DOM-based Cross-Site Scripting (XSS) vulnerability was also confirmed in the application's search functionality. Additional findings relate to insecure data transmission and missing HTTP security headers, identified through manual network inspection and automated scanning respectively.

These issues are representative of common real-world weaknesses caused by insufficient input validation, absent output encoding, and incomplete security header configuration. Detailed reproduction steps, evidence, and remediation guidance for each finding are provided in the sections that follow.

Scope & Methodology

The assessment was limited to the locally hosted instance of the OWASP Juice Shop application running on port 3000. The following areas were explored: account registration, login/authentication, and the product search feature.

Testing Approach

- Application reconnaissance — exploring signup, login, and core navigation flows.
- Manual Cross-Site Scripting (XSS) testing using crafted payloads injected into input fields.
- Manual SQL Injection testing against the authentication form.

- Inspection of live network traffic via Browser Developer Tools to assess how sensitive data (e.g. passwords) is transmitted and returned by the server.
- Automated vulnerability scanning using OWASP ZAP (Spider crawl followed by an Active Scan).

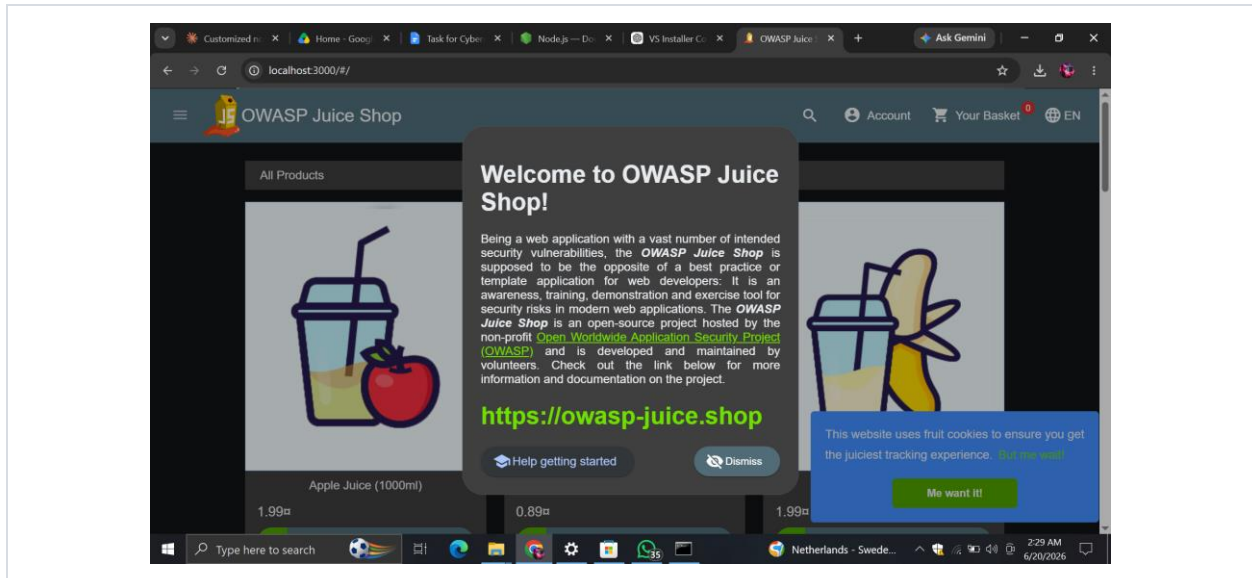


Figure 1 — Target application confirmed running at <http://localhost:3000>.

Summary of Findings

#	Vulnerability	Severity	Status
1	SQL Injection — Authentication Bypass	CRITICAL	Confirmed
2	Cross-Site Scripting (DOM-based)	HIGH	Confirmed
3	Password Transmitted Over Unencrypted Connection	MEDIUM	Confirmed
4	Missing Content-Security-Policy Header	MEDIUM	Confirmed
5	CORS Misconfiguration — Wildcard Origin	MEDIUM	Confirmed
6	Server Timestamp Disclosure	LOW	Confirmed

Detailed Findings

F1. SQL Injection — Authentication Bypass

CRITICAL**Location:** Login form — POST /rest/user/login

Description

The login form's email field does not properly sanitize or parameterize user input before incorporating it into the backend SQL query. This allows an attacker to inject SQL syntax that alters the logic of the authentication query, bypassing the login mechanism entirely.

Steps to Reproduce

1. Navigate to the application login page.
2. Enter the following payload into the Email field:

```
' OR 1=1--
```

3. Enter any arbitrary value into the Password field.
4. Submit the login form.

Result

The application authenticated the session without valid credentials and granted access to an administrator account. This was independently confirmed by the application itself, which flagged the action as the “Login Admin” security challenge being solved.

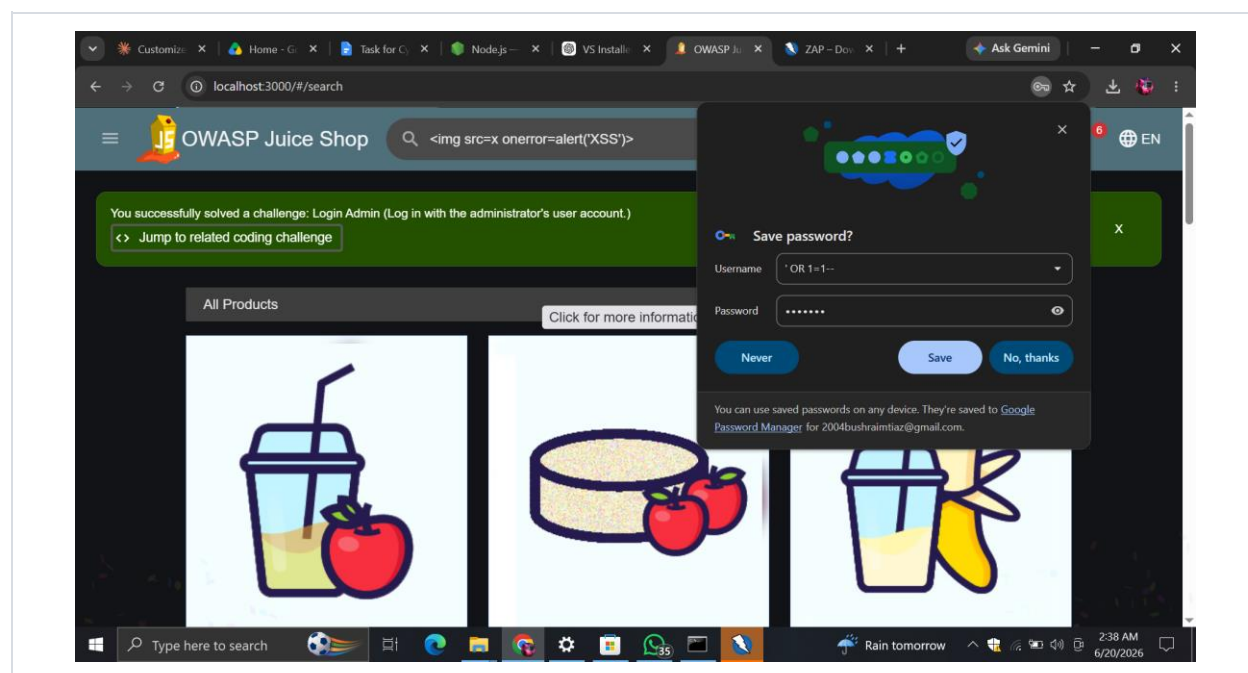


Figure 2 — Successful authentication bypass using SQL Injection payload.

Impact

An attacker requires no valid credentials to gain full administrative access to the application. This could lead to complete account takeover, unauthorized data access, and compromise of all user records managed by the system.

Recommendation

Replace raw SQL string concatenation with parameterized queries or prepared statements (or use an ORM that handles escaping automatically). Apply strict server-side input validation on all authentication fields, and ensure the same protection is applied consistently across every database query in the application.

F2. Cross-Site Scripting (DOM-based)

HIGH

Location: Product search field — `/#/search`

Description

The search functionality reflects user-supplied input directly into the page DOM without adequate sanitization or output encoding. While a standard `<script>` tag payload was blocked by the browser, an image-tag-based payload using an `onerror` event handler successfully bypassed this restriction and executed arbitrary JavaScript.

Steps to Reproduce

5. Navigate to the product search bar.
6. Enter the following payload:

```
<img src=x onerror=alert('XSS')>
```

7. Submit the search query.

Result

The injected JavaScript executed successfully, triggering a browser alert dialog displaying “XSS”. This confirms the application does not properly sanitize input that is rendered into the DOM.

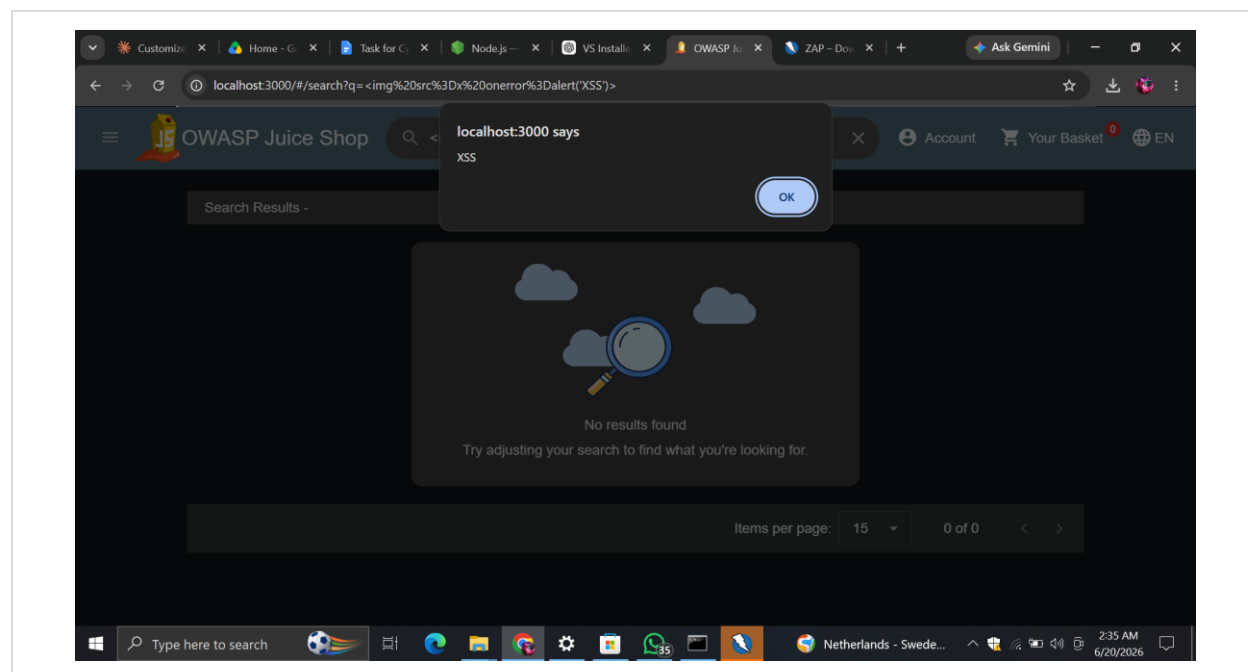


Figure 3 — JavaScript execution confirming DOM-based XSS via the search field.

Impact

An attacker could exploit this vulnerability to execute arbitrary scripts in a victim's browser session, enabling session hijacking, credential theft, defacement, or redirection to malicious websites — particularly if the payload is delivered via a shared or crafted URL.

Recommendation

Sanitize and encode all user-supplied input before it is rendered into the DOM. Use a front-end framework that auto-escapes output by default, and implement a strict Content-Security-Policy to restrict which scripts are permitted to execute (see Finding F4).

F3. Password Transmitted Over Unencrypted Connection

MEDIUM

Location: POST /api/Users/ (Registration), POST /rest/user/login

Description

The application is served over plain HTTP rather than HTTPS. Inspection of network traffic during account registration confirmed that the password value is transmitted from the client to the server in plaintext, with no transport-layer encryption protecting it in transit.

Steps to Reproduce

8. Open Browser Developer Tools and select the Network tab.
9. Complete the account registration form.
10. Locate the resulting request to the Users endpoint and inspect its payload.

Result

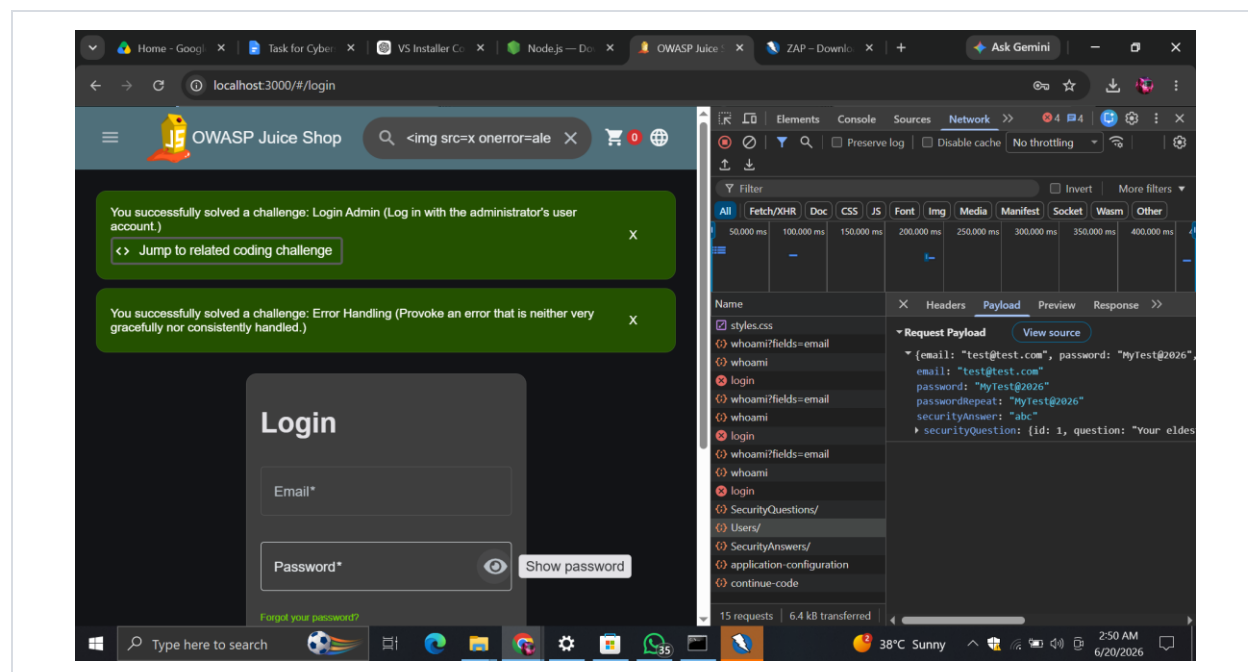


Figure 4 — Registration request payload showing the password transmitted in plaintext.

On the positive side, inspection of the corresponding server response confirmed that the password value is not echoed back to the client — only non-sensitive account fields (id, email, role, etc.) are returned.

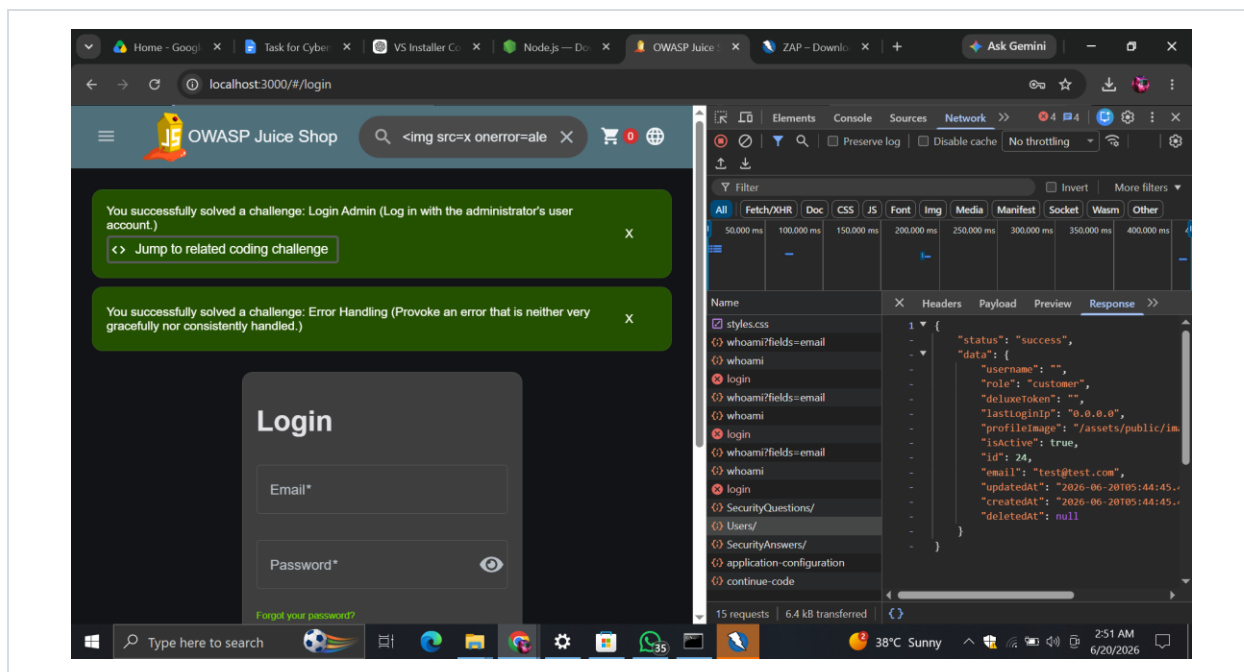


Figure 5 — Server response confirming the password is not exposed back to the client.

Impact

Without TLS/HTTPS, credentials transmitted across the network are vulnerable to interception via a Man-in-the-Middle (MITM) attack, particularly on untrusted networks such as public Wi-Fi.

Recommendation

Enforce HTTPS across the entire application using a valid TLS certificate, redirect all HTTP requests to HTTPS, and enable HTTP Strict Transport Security (HSTS) to prevent protocol downgrade attacks.

F4. Missing Content-Security-Policy (CSP) Header

MEDIUM

Location: Application-wide — all HTTP responses

Description

Automated scanning with OWASP ZAP identified that the application does not set a Content-Security-Policy response header on its pages.

Impact

Without a CSP, the browser has no policy restricting which scripts, styles, or resources are permitted to load or execute. This increases the potential impact of the Cross-Site Scripting vulnerability identified in Finding F2.

Recommendation

Implement a Content-Security-Policy header (for example, via the helmet.js middleware in an Express application) restricting script-src, style-src, and other resource directives to explicitly trusted origins.

F5. CORS Misconfiguration — Wildcard Origin

MEDIUM

Location: Application-wide — API responses

Description

Response headers captured during testing show an Access-Control-Allow-Origin value of “*”, meaning the API accepts cross-origin requests from any domain without restriction.

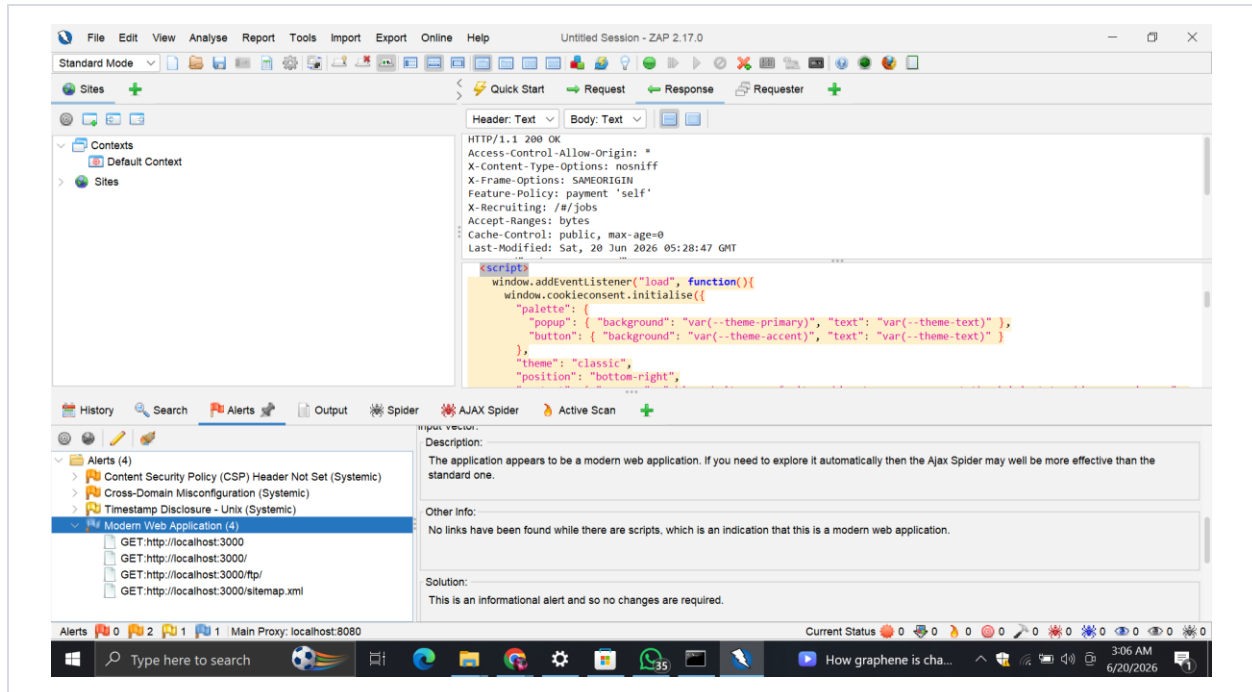


Figure 6 — OWASP ZAP automated scan results, including response headers showing the wildcard CORS policy, missing CSP header, and additional informational findings.

Impact

A wildcard CORS policy can allow a malicious website to make authenticated cross-origin requests on behalf of a logged-in user and read the response, increasing the risk of cross-site data theft.

Recommendation

Restrict Access-Control-Allow-Origin to an explicit whitelist of trusted domains rather than using a wildcard, particularly for any endpoint that returns authenticated or sensitive data.

F6. Server Timestamp Disclosure

LOW

Location: Application-wide — HTTP responses

Description

OWASP ZAP's automated scan flagged the disclosure of Unix-formatted timestamps within application responses (classified by ZAP as a systemic, low-risk informational finding).

Impact

The risk on its own is minimal, but exposed timestamps can assist an attacker during reconnaissance, for example in fingerprinting server behavior, caching logic, or timezone configuration.

Recommendation

Where not required by the client application, strip internal timestamp fields from API responses, or restrict their precision/format to reduce information disclosure.

Areas of Improvement

In addition to remediating the specific findings above, the following general improvements are recommended to strengthen the application's overall security posture:

- Apply consistent input validation and output encoding across all user-input fields to systematically prevent injection and XSS-class vulnerabilities.
- Replace all instances of raw SQL string concatenation with parameterized queries or an ORM throughout the codebase, not only the login form.
- Enforce HTTPS/TLS for all traffic and disable plain HTTP access in any production-equivalent environment.
- Configure standard security headers (Content-Security-Policy, X-Frame-Options, X-Content-Type-Options) using a middleware such as helmet.js.
- Restrict the CORS policy to a defined set of trusted origins instead of a wildcard.
- Incorporate periodic automated (e.g. OWASP ZAP) and manual security assessments into the regular development lifecycle.

Conclusion

This Week 1 security assessment of the OWASP Juice Shop User Management System identified six confirmed findings: one Critical, one High, three Medium, and one Low severity. While OWASP Juice Shop is intentionally designed to be vulnerable for training purposes, each issue identified here mirrors a common real-world weakness — unsanitized input leading to injection and XSS, unencrypted credential transmission, and incomplete HTTP security header configuration. Implementing the recommendations outlined in this report would meaningfully improve the application's resistance to these classes of attack.